clasp: sync code + run the integration test from the command line

clasp is Google's Apps Script CLI. It replaces the "copy each file from GitHub and paste into the Apps Script editor" dance with a single `npm run clasp:push`, and lets us run the real-backend integration test (`runIntegrationSmoke`) without opening the editor.

Everything that doesn't need your Google login is already set up (clasp installed, `.claspignore` whitelist, `.clasp.json`, npm scripts). The steps below are the ones **only you** can do, because they authenticate as your own Google account.

Run all commands from `products/service-crm/`.

Part 1: Code sync (the main win, ~5 min)

1. Turn on the Apps Script API for your account (one time). Open https://script.google.com/home/usersettings → toggle **Google Apps Script API = ON**.

2. Get your Script ID and put it in `.clasp.json`. In the Apps Script editor (Extensions ▸ Apps Script) → **Project Settings** (gear icon) → copy the **Script ID**. Paste it into `.clasp.json`, replacing `PASTE_YOUR_SCRIPT_ID_HERE`. (Or just paste the Script ID to me and I'll drop it in.)

3. Log in.

```
npm run clasp:login
```

A browser opens; sign in with **the Google account that owns the CRM sheet/script** (must be the same account, not a different one), and allow the permissions. This stores a token in `~/.clasprc.json` (gitignored, never committed).

4. Confirm clasp will push ONLY the 8 project files.

```
npm run clasp:status
```

You should see exactly: `appsscript.json, Code.gs, Api.gs, db.gs, setup.gs, Tests.gs, WebApp.html, Sidebar.html`, and nothing else (no tests, no .md, no other .html). If anything extra shows up, stop and tell me.

5. Push the code.

```
npm run clasp:push
```

This uploads the current files to your Apps Script project, the same thing you were doing by hand, now in one command. From here on, after I make a change: `git pull` (or grab the files) → `npm run clasp:push` → done. Re-run **Set up / rebuild database** in the sheet only when the schema changed (db.gs / setup.gs).

> `npm run clasp:pull` does the reverse (project → local) if you ever edit in the web editor.

Part 2: Run the integration test from the CLI (optional, more setup)

`clasp run` executes a function in your project remotely. It needs a bit more wiring:

A. Attach a standard Google Cloud project. Apps Script editor ▸ Project Settings ▸ **Google Cloud Platform (GCP) Project** ▸ Change project → paste a **GCP project number**. If you don't have one: https://console.cloud.google.com/ → create a project → copy its number. (On the same project, make sure the **Apps Script API** is enabled.)

B. Declare the scopes + execution API. Tell me when Part A is done, and I'll add the `oauthScopes` and `executionApi` block to `appsscript.json` and push it (clasp needs the manifest to expose the function).

C. Re-login once so the token carries the new scopes, then run:

```
npm run clasp:smoke      # = clasp run-function runIntegrationSmoke
npm run clasp:logs       # see the pass/fail output
```

Once Part 2 is set up, **I can run the real Sheet + Calendar integration test for you** on each change (you'll still approve the Calendar permission the first time). Until then, run `runIntegrationSmoke` from the editor (SMOKE-TEST.md Section O).

Troubleshooting

- **"User has not enabled the Apps Script API"** → do Part 1 step 1, wait a minute, retry.
- `clasp:status` **lists extra files** → the `.claspignore` whitelist didn't match; don't push, tell me.
- **Wrong account** → `clasp logout` then `npm run clasp:login` with the account that owns the sheet.
- `clasp run` **"not found / not deployed"** → Part 2 not finished (GCP project + manifest scopes).